Ders 4 Genişletilmiş Notu: Deterministik Olmayan Sonlu Otomatlar, Hesaplama Ağaçları ve NFA-DFA Denkliği

Ders 4 Genişletilmiş Notu: Deterministik Olmayan Sonlu Otomatlar, Hesaplama Ağaçları ve NFA-DFA Denkliği

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. DFA'dan NFA'ya: İlk İki Fark
- 2. NFA'nın Biçimsel Tanımı
- 3. Bir NFA Dizgiyi Nasıl İşler?
- 4. Hesaplama Ağacı
- 5. Dil Bulma Örnekleri
- 6. NFA Daha Kolaydır, Ama Daha Güçlü Değildir
- 7. Genişletilmiş Geçiş Fonksiyonu
- 8. Bir NFA'nın Dili
- 9. Altı Durumlu Örnek
- 10. Makinelerin Denkliği
- 11. Teorem: NFA'lar Düzenli Dilleri Kabul Eder
- 12. NFA'dan DFA'ya Dönüşüm

1. DFA'dan NFA'ya: İlk İki Fark

Bu hafta sonlu otomatlarla (finite automata) devam ediyoruz ve deterministik olmayan sonlu otomatları (nondeterministic finite automata) öğrenmeye başlıyoruz. Hoca bundan sonra uzun adlar yerine kısaltma kullanacağını söyledi: deterministik olmayan sonlu otomat için NFA, deterministik sonlu otomat (deterministic finite automata) için DFA. İki ad arasındaki fark "non" ekidir. Hoca, adların neden böyle olduğunu örneklerle göreceğimizi söyledi.

1.1 DFA'yı Hatırlayalım

Bir DFA'da durumlar vardır. Alfabemiz $\{a, b\}$ olsun. Her durum ve alfabedeki her sembol için **bir ve yalnız bir** geçiş bulunur. Yani her durum için `a` gelince ne olacağını da, `b` gelince ne olacağını da söylemek zorundayız. Bu alfabede her durumdan tam olarak iki geçiş çıkar. NFA'lar bu kuralı üç noktada gevşetir. İlk ikisi bu bölümde, üçüncüsü Bölüm 2.3'tedir.

1.2 Birinci Fark: Aynı Sembolle Birden Çok Geçiş

NFA'da aynı sembol için birden çok geçiş olabilir. Örneğin q_1 durumundayken `a` geldiğinde hem q_2'ye hem de q_3'e gidilebilir:

$$q_1 \xrightarrow{a} q_2, \qquad q_1 \xrightarrow{a} q_3$$

DFA'da bu olamaz. Orada `a` ile yalnız tek bir duruma gidilir.

1.3 İkinci Fark: Boş Geçiş

NFA'da boş geçiş (null transition) olabilir. Şu makineyi düşünelim:

$$q_0 \xrightarrow{a} q_1 \xrightarrow{\varepsilon} q_2$$

Buradaki ε sembolü "null" diye okunur. Bazı yerlerde onun yerine λ görebilirsiniz. İkisi de aynı şeydir. ε hiçbir zaman alfabede yer almaz. Burada alfabe örneğin $\{a, b\}$ olabilir, ama içinde ε göremezsiniz.

Bu geçişin söylediği şudur: q_0'da a okununca makine yalnız q_1'e değil, aynı anda q_2'ye de geçer. Adım adım bakalım:

1. Başta makine q_0 durumundadır.
2. a sembolü gelince q_0'dan q_1'e geçilir.
3. Başka bir giriş beklenmeden q_1'den q_2'ye de geçilir.
4. Artık iki durum da etkindir (active). Bundan sonra gelen her sembolü ikisi de işlemeye devam eder.

Aynı anda birden çok durumda olmak ilk başta kafa karıştırabilir. Hocanın önerdiği sezgi paralel hesaplamadır (parallel computation): Böyle bir geçişte makine **kendini çoğaltır** ve paralel çalışan iki makine olur. Hoca bunlara bir an "sanal makine" (virtual machine) dedi, ama bu sözü kendisinin uydurduğunu söyleyip tahtadan sildi. Bunun yerine iki yürütme yolundan (execution path) ya da iki hesaplama dalından (computation branch) söz etti.

2. NFA'nın Biçimsel Tanımı

Tanım DFA'nınkine çok benzer. NFA bir beşlidir (5-tuple):

$$M = (Q, \Sigma, \delta, q_0, F)$$

- Q: bütün durumların kümesi
- Σ: alfabe
- δ: geçiş fonksiyonu (transition function)
- q_0: başlangıç durumu (initial state)
- F: kabul durumlarının kümesi (set of accepting states)

2.1 Bir Örnek

Hocanın tahtaya çizdiği makinenin geçişleri şunlardır:

- $q_0 \xrightarrow{1} q_1$
- $q_1 \xrightarrow{0} q_0$
- $q_1 \xrightarrow{0,1} q_2$
- $q_0 \xrightarrow{\varepsilon} q_2$

Bu makinede $Q = \{q_0, q_1, q_2\}$ ve $\Sigma = \{0, 1\}$ olur. Yani girişte yalnız 0 ve 1 bulunabilir. Başlangıç durumu q_0'dır. Yalnız q_0 kabul eder, dolayısıyla $F = \{q_0\}$.

Fark geçiş fonksiyonundadır. Bir sembol için birden çok duruma gidilebildiğinden fonksiyonun değeri tek bir durum olamaz. δ'yı yazmanın farklı yolları vardır. Hoca tablo seçti ve önce 0 ile 1 sütunlarını doldurdu:

δ	0	1
q_0	$\emptyset$	$\{q_1\}$
q_1	$\{q_0, q_2\}$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$

Satırları şöyle okuruz. q_0'dayken 0 gelirse gidilecek yer yoktur, yani hiç geçiş yoktur. Bunu boş küme ile yazarız. q_1'deyken 0 gelirse hem q_0'a hem q_2'ye gidilebilir, ikisi de mümkündür. Bu yüzden $\delta(q_1, 0) = \{q_0, q_2\}$ yazılır. q_2'den hiçbir sembolle geçiş yoktur.

Tablonun bir ayrıntısına dikkat edin: Gidilebilecek tek durum olsa bile onu küme içinde yazarız. q_0'da 1 gelince tek hedef q_1'dir, ama değer $\{q_1\}$'dir.

2.2 Geçiş Fonksiyonu ve Kuvvet Kümesi

Hoca δ'nın neyi alıp neyi verdiğini de yazdı. Fonksiyon Q'dan bir durum ve alfabeden bir sembol alır, sonuç olarak bir durum kümesi verir. Bu kümelerin hepsi Q'nun kuvvet kümesinin (power set) elemanıdır:

$$\delta : Q \times \Sigma \to 2^Q$$

> **Ders sonrası not:** Hoca tanım kümesini bu noktada $Q \times \Sigma$ olarak söyledi. Bölüm 2.4'te tabloya ε sütunu da eklendi. Boş geçişleri de kapsayan doğru yazım $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$ biçimindedir.

Bu örnekte Q'nun üç elemanı vardır. Kuvvet kümesinin elemanları şöyle sayılır:

- Boş küme: $\emptyset$
- Bir elemanlı kümeler: $\{q_0\}$, $\{q_1\}$, $\{q_2\}$
- İki elemanlı kümeler: $\{q_0, q_1\}$, $\{q_0, q_2\}$, $\{q_1, q_2\}$
- Üç elemanlı küme: $\{q_0, q_1, q_2\}$

Toplam 8 küme vardır. Geçiş fonksiyonu her seferinde bunlardan birini verir.

2.3 Üçüncü Fark: Yazılmayan Geçişler

NFA'da bir sembol için ne olacağını hiç yazmamak mümkündür. Alfabe $\{a, b\}$ olsun. Yalnız şu geçişin bulunduğu bir q_0 durumu NFA için geçerlidir:

$$q_0 \xrightarrow{a} q_1$$

Burada b geldiğinde ne olacağı yazılmamıştır. Böyle bir geçiş yazılmamışsa ve girişte o sembol gelirse, **o hesaplama dalı (execution branch) o sembolde reddeder.**

Böylece NFA ile DFA arasındaki üç fark tamamlanır:

- Bir sembolle birden çok duruma gidilebilir.
- Boş geçişler olabilir.
- Bir sembol için geçiş yazılmayabilir.

2.4 Tabloda Boş Geçişler

Soru: Geçiş fonksiyonu tablosunda boş geçişi nasıl göstereceğiz?

Cevap: Hoca önce bu tabloda boş geçişin örtük (implicit) olduğunu, doğrudan görünmediğini söyledi.

Soru: O zaman geçiş tablosu verildiğinde makinenin grafını çizemeyiz, değil mi?

Cevap: Hoca soruyu sorana hak verdi. Tablo makineyi tam tanımlamalıysa ε da yazılmalıdır, çünkü q_0'dan q_2'ye bir ε geçişi vardır. Tabloya giriş olarak bir ε sütunu ekledi.

Bu sütunda ilginç değeri olan tek hücre vardır: q_0'dayken giriş olmadan q_2'ye gidilir. q_1 ve q_2'de giriş olmadan bir şey olmaz. Hoca $\emptyset$ değerini burada "bir şey olmaz" diye okumanın daha iyi olduğunu söyledi. Tablonun son hali şöyledir:

δ	0	1	ε
q_0	$\emptyset$	$\{q_1\}$	$\{q_2\}$

δ	0	1	ε
q_1	$\{q_0, q_2\}$	$\{q_2\}$	$\emptyset$
q_2	$\emptyset$	$\emptyset$	$\emptyset$

3. Bir NFA Dizgiyi Nasıl İşler?

NFA'nın girişi nasıl tükettiğini (consume) bir örnekle görelim. Makine dört durumludur, tek kabul durumu q_2'dir:

$$q_0 \xrightarrow{a} q_1, \qquad q_1 \xrightarrow{a} q_2, \qquad q_0 \xrightarrow{a} q_3$$

Hoca, NFA'nın çalışmasını anlamak için hesaplama ağacını (computation tree) çizmeyi çok faydalı bulduğunu söyledi. Bu makine basit olduğu için ağaç makinenin kendisine benziyor, ama yine de ağaçla başladı.

3.1 `aa` Dizgisi

Başta q_0 durumundayız. İlk `a` okununca q_0 iki duruma geçer: q_1 ve q_3. Makine kendini çoğaltmış gibidir ve artık iki hesaplama dalımız vardır. İkinci `a`'yı tüketmeye çalışalım:

- q_1 bu `a`'yı tüketebilir ve kabul durumu olan q_2'ye geçer. Dizgi burada biter, okunacak sembol kalmaz. Bu dal kabul eder.
- q_3 bu `a`'yı tüketemez, çünkü q_3'ten `a` geçişi yoktur. Bu dal reddeder.

```
q0 --a--> q1 --a--> q2   (giriş bitti, kabul durumu: kabul)
q0 --a--> q3 --a--> yok  (sembol tüketilemedi: ret)
```

Bir dal reddetti, bir dal kabul etti. Makinenin kararı nedir? Makine `aa`'yı **kabul eder**, çünkü en az bir kabul eden dal vardır.

Hoca burada adlandırmayı da netleştirdi: Bu dalların her birine hesaplama (computation) denir. Yani `aa` için iki hesaplama vardır.

> **Hocanın vurgusu:** Bir NFA, o dizgiyi kabul eden bir hesaplama varsa dizgiyi kabul eder. Gereken tek şey böyle tek bir hesaplamanın var olmasıdır.

3.2 `a` Dizgisi

Aynı makineye yalnız `a` verelim. q_0, `a` ile q_1 ve q_3'e geçer. Okunacak sembol kalmamıştır. İki hesaplama da kabul olmayan bir durumdadır, ikisi de reddeder ve makine `a`'yı reddeder.

İki örnekte reddetme sebebi farklıdır. `a` dizgisinde iki hesaplama da bütün girişi okuyup kabul olmayan bir durumda biter. `aa` dizgisindeki q_3 dalında ise sembol tüketilemez.

3.3 Ret Koşulu

Bir NFA, dizgiyi kabul eden hiçbir hesaplama yoksa dizgiyi reddeder. Bunun olması için **her hesaplamada** şu iki durumdan biri gerçekleşmelidir:

- Bütün giriş tüketilmiştir ve otomat kabul olmayan bir durumdadır. `a` dizgisindeki durum budur.
- Giriş tüketilemez. `aa` dizgisindeki q_3 dalında olan budur.

4. Hesaplama Ağacı

Bir NFA ve bir w dizgisi verildiğinde hesaplama ağacı, bütün hesaplamaları görmemizi sağlar.

> **Hocanın vurgusu:** Karmaşık NFA'larla uğraşırken hesaplama ağacı çok faydalıdır. Basit makinelerde gerekmeyebilir, ama hoca karmaşık NFA'larda bu yöntemi çok sevdiğini söyledi.

5. Dil Bulma Örnekleri

5.1 $q_0 \xrightarrow{a} q_1 \xrightarrow{\varepsilon} q_2 \xrightarrow{a} q_3$

Tek kabul durumu q_3'tür. q_0'dan `a` ile q_1'e geçilir. Boş geçiş bizi doğrudan q_2'ye götürür. Sonra bir `a` daha gelirse q_3'e ulaşır ve kabul ederiz. Başka bir olasılık yoktur, bu yüzden:

$$L(M) = \{aa\}$$

Başka dizgilerde ne olduğuna bakalım:

- `a`: Hesaplama q_1'de kalır ve q_1 reddeden bir durumdur.
- `aaa`: İkinci `a` ile q_3'e geliriz, ama son `a` için q_3'te geçiş yoktur. Dizgi reddedilir.

Bir makinenin dizgiyi reddettiği iki durum burada da görülür.

> **Ders sonrası not:** Hoca `a` için hesaplamanın q_1'de durduğunu söyledi. Boş geçiş nedeniyle $\delta^*(q_0, a) = \{q_1, q_2\}$ olur, yani q_2 dalı da etkindir. q_2 de kabul durumu olmadığı için sonuç değişmez. Python simülasyonu 10 sembole kadar `a` dizgilerinde $L(M) = \{aa\}$ sonucunu verdi.

Soru: Bu örnekte alfabe yalnız $\{a\}$ mı, yoksa ε de alfabede mi?

Cevap: ε hiçbir zaman alfabede değildir, bu derste böyle anlaştık. Özel bir semboldür. Buradaki alfabe yalnız $\{a\}$'dır.

> **Hocanın vurgusu:** ε (null) hiçbir zaman alfabede değildir. Bu dersteki özel bir sembol ya da karakterdir.

5.2 Döngülü Örnek ve $(ab)^+$

Dört durumlu makinenin geçişleri şunlardır:

$$q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2 \xrightarrow{\varepsilon} q_3 \xrightarrow{\varepsilon} q_0$$

> **Hocanın vurgusu:** Başlangıç durumunu işaretlemeyi unutmayın. Hoca çizimlerinde iki kez bu işareti atladı ve "siz çizerken unutmayın" dedi.

q_0'dan `a` ile q_1'e, `b` ile q_2'ye geçilir. q_2 kabul durumudur, dolayısıyla `ab` kesinlikle kabul edilir.

Sonra başka girdi olmadan önce boş geçişle q_3'e, hemen ardından yine girdi olmadan q_0'a geçilir. Başlangıca döndük, yani bir döngü (loop) görüyoruz. Bu yüzden makine çok uzun dizgileri de kabul edebilir. q_0'a döndüğümüzde yeni bir `ab` okuyabilir, yine q_2'ye ulaşır, yine q_0'a dönebiliriz. Desen ortadadır.

Dili yazarken hoca önce $(ab)^*$ demeye başladı, ama hemen vazgeçti. Yıldız kullanılamaz, çünkü boş dizgi kabul edilmiyor: Başlangıç durumu kabul durumu değildir. Boş dizgi olsaydı yıldız yazabilirdik. Burada en az bir `ab` gereklidir ve bu, artı işaretiyle yazılır:

$$L(M) = (ab)^+ = \{ab, abab, ababab, \dots\}$$

> **Ders sonrası not:** Hoca makinenin "sonsuz uzunlukta" (infinitely long) dizgileri kabul edebildiğini söyledi. Doğrusu, her dizgi sonludur. Dildeki dizgilerin uzunluğunun bir üst sınırı yoktur, dil sonsuzdur. Python simülasyonu 14 sembole kadar bütün $\{a, b\}$ dizgilerinde $L(M) = (ab)^+$ sonucunu verdi.

Soru: (Öğrenci gösterimde gördüğü parantez biçimini sordu.) Böyle bir yazım mümkün mü?

Cevap: Evet, mümkündür. Normal parantez gruplama için kullanılır. Her ikisi de kullanılır, kümeler için de kullanılır.

5.3 $(10)^*$

Bu makine, Bölüm 2.1'deki makinedir: $q_0 \xrightarrow{1} q_1$, $q_1 \xrightarrow{0} q_0$, $q_1 \xrightarrow{0,1} q_2$, $q_0 \xrightarrow{\varepsilon} q_2$. Başlangıç durumu q_0'dır ve q_0 kabul eder.

Baştan itibaren izleyelim:

- Başlangıç kabul durumu olduğu için boş dizgi kabul edilir.
- `1` gelince q_1'e geçilir.
- Ardından `0` gelince ya q_0'a dönülür ya da q_2'ye gidilir.
- q_0 kabul durumudur, dolayısıyla `10` kabul edilir.
- q_2 kabul durumu değildir ve ondan çıkan geçiş yoktur. Oraya giden hesaplama durur.

Bu nedenle q_2 bir ölü durumdur (death state), hoca bunu önceki derslerden hatırlattı. Dahası q_2 gereksiz bir durumdur (redundant state): Bu durumu makineye hiç koymasak da hiçbir şey değişmezdi.

q_0'a döndükten sonra girişi işlemeye devam edebiliriz. Bir `1` ve bir `0` daha okuyabilir, `1010` elde edebiliriz. Desen böyle sürer. Bu kez artı yerine yıldız kullanırız, çünkü boş dizgi de dildedir. Yıldız, hiç tekrar olmamasına da izin verir:

$$L(M) = (10)^*$$

Hoca, işler karmaşıklaştığında hesaplama ağacı çizmeyi faydalı bulduğunu söyleyip bu makine için ağaç çizdi. Normalde ağacı bir w dizgisi üzerinde çizeriz. Burada `1010` benzeri bir dizgiye bakıldı.

```
q0 --1--> q1 --0--> q0 --1--> q1 ... (döngü sürer)
             --0--> q2 --1--> yok, durur, ret
```

Hoca burada döngüyü gösterdi: Ağaçta yeniden başa dönmüş gibi oluruz. Soruya göre gerekli olur ya da olmaz, ama bazen hesaplama ağaçları işi çok kolaylaştırır.

> **Ders sonrası not:** Hoca ağaçta q_0'dan çıkan ε dalını çizmedi. Tam ağaçta q_0'a her gelindiğinde q_2 dalı da açılır. Etkin durumlar `1010` için simülasyonla aşağıdaki gibi bulundu (ara adım eklendi). q_2 dalları sonucu değiştirmez. Python simülasyonu, ε dalı dahil, 14 sembole kadar bütün $\{0,1\}$ dizgilerinde $L(M) = (10)^*$ verdi. q_2 silinince de aynı dil çıktı.

Okunan önek	Etkin durumlar
ε	$\{q_0, q_2\}$
`1`	$\{q_1\}$
`10`	$\{q_0, q_2\}$
`101`	$\{q_1\}$
`1010`	$\{q_0, q_2\}$

6. NFA Daha Kolaydır, Ama Daha Güçlü Değildir

Hoca örneklerin ardından iki gözlem yazdı. İlki bildiğimiz şeydir: ε alfabede değildir. İkincisi şudur: NFA'lar bir dili ifade etmek için daha kolaydır.

> **Hocanın vurgusu:** "Daha kolay", "daha güçlü" demek değildir. NFA'lar DFA'lardan daha güçlü değildir. NFA'larla kabul edilebilen diller, DFA'larla kabul edilebilen dillerin tam olarak aynısıdır.
>
> **Ders sonrası not:** Hoca bu cümleyi söylerken önce "NFA'larla kabul edilen diller, NFA'larla kabul edilen dillerle aynıdır" dedi, hemen ardından DFA'yı söyledi. Doğrusu yukarıdaki biçimdir.

Neden daha kolay? Şu NFA'ya, yani M_1'e bakalım:

$$q_0 \xrightarrow{a} q_1 \quad (q_1 \text{ kabul durumu})$$

Dili yalnız a dizgisinden oluşur: $L(M_1) = \{a\}$. Şimdi buna karşılık gelen bir DFA olan M_2'yi yazmaya çalışalım. İstediğimiz şey $L(M_1) = L(M_2)$ olmasıdır.

DFA'da her durum ve her sembol için geçişi göstermek zorundayız. Örneğin q_1'de a gelince ne olacağını yazmalıyız. Bu geçişi geri kabul durumuna döndürürsek başka a'lı dizgileri de kabul etmiş oluruz, bunu yapamayız. Bu yüzden yalnız kuralları sağlamak için yeni bir q_2 durumu eklemek gerekir ve bu durum kabul durumu olmaz. Sonra q_2'nin kendisi için de alfabedeki her sembolde ne olacağını yazmamız gerekir. Sonuçta makine şöyle olur (ara adım eklendi):

Durum	a	b
q_0 (başlangıç)	q_1	q_2
q_1 (kabul)	q_2	q_2
q_2	q_2	q_2

Python simülasyonu 12 sembole kadar bütün $\{a, b\}$ dizgilerinde bu DFA'nın da NFA'nın da yalnız a dizgisini kabul ettiğini gösterdi. Fazladan durum ve geçişler yalnız DFA'nın kurallarına uymak içindir. NFA'da bu kurallar daha esnektir.

7. Genişletilmiş Geçiş Fonksiyonu

DFA'da olduğu gibi NFA'da da genişletilmiş geçiş fonksiyonu (extended transition function) vardır. Normal geçiş fonksiyonuna benzer, ama yıldızla yazılır ve sembol değil dizgi alır. Bir sembol için birden çok duruma gidilebildiği gibi, bir dizgi için de birden çok duruma gidilebilir. Bu yüzden sonuç yine bir kümedir. Hocanın gösterim örneği şöyleydi:

$$\delta^*(q_0, aa) = \{q_4, q_5\}$$

Genel olarak şunu söyleriz:

$$q_j \in \delta^*(q_i, w) \iff q_i\text{'den } q_j\text{'ye } w \text{ etiketli bir yürüyüş (walk) vardır}$$

Bunu kısaca $q_i \xrightarrow{w} q_j$ diye gösteririz. Yürüyüşün ayrıntıları bu gösterimde verilmez. Yalnız şunu söyler: Dizgi $w = \sigma_1\sigma_2 \ldots \sigma_k$ ise, σ_1 ile bir duruma, ardından σ_2 ile başka bir duruma gidilir, böylece sürer ve son sembol σ_k ile q_j'ye varılır.

> **Ek bilgi (derste anlatılmadı):** Bu yürüyüşte ε etiketli kenarlar da kullanılabilir ve etikete bir şey eklemez. Bu yüzden $\delta^*(q_i, w)$ hesaplanırken ε geçişleriyle ulaşılan durumlar da kümeye dahil edilir. Örneğin Bölüm 5.2'deki makinede $\delta^*(q_0, ab) = \{q_2, q_3, q_0\}$ olur.

8. Bir NFA'nın Dili

Şimdiye kadar NFA'ların dillerine zaten baktık. Hoca bunu daha biçimsel yazmaya çalıştı. Yazdığı tanımı şöyle özetleyebiliriz: $L(M)$, $w_0, w_1, \ldots, w_n$ dizgilerinden oluşan kümedir. Bu dizgilerden herhangi biri w_m için başlangıçtan yola çıkılınca bir durum kümesi elde edilir:

$$\delta^*(q_0, w_m) = \{q_i, q_k, q_j\}$$

Bu kümedeki durumlardan **en az biri** kabul durumudur, yani $q_k \in F$ olan bir q_k vardır. F'nin kabul durumlarının kümesi olduğunu hatırlayalım.

Soru: Dildeki dizgileri w_n'e kadar yazdınız, aşağıda w_m kullandınız. Bu bir yanlış mı, ikisi farklı mı?

Cevap: Hoca bu biçimsel yazımın tam olmadığını, üzerinde biraz daha uğraşmak gerektiğini söyledi. m bir indekstir ve aradaki değerlerden biridir. Tanıma $0 \leq m \leq n$ koşulunu ekledi.

Hocanın vurgusu: Biçimsel tanımları her şeyi kapsayacak kadar kesin yazmak için çok dikkatli çalışmak gerekir.

Ders sonrası not: Dizgileri $w_0, \ldots, w_n$ diye sıralamak dilin sonlu olduğunu düşündürür. Oysa $(ab)^+$ gibi diller sonsuzdur. Hocanın anlatmak istediği şeyin kesin yazımı şudur: $L(M) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \cap F \neq \emptyset\}$.

9. Altı Durumlu Örnek

Biraz daha karmaşık bir makineye bakalım. Durumlar q_0'dan q_5'e kadardır. Başlangıç durumu q_0'dır. Kabul durumları q_0 ve q_5'tir. Alfabe $\{a, b\}$'dir. Geçişler kısa nottaki çizime göre şöyledir:

- $q_0 \xrightarrow{a} q_1$
- $q_1 \xrightarrow{b} q_2 \xrightarrow{\varepsilon} q_3 \xrightarrow{\varepsilon} q_0$
- $q_1 \xrightarrow{a} q_4$
- $q_1 \xrightarrow{a} q_5$

Dili bulalım. Başta hiç giriş almadan kabul durumundayız, dolayısıyla $\varepsilon \in L(M)$. q_0 dışında tek kabul durumu q_5'tir ve oraya `a`'yı izleyen bir `a` ile gidilir. Başa dönen bir döngü de görüyoruz. Demek ki dil sonsuzdur. Bütün elemanları yazamayacağımız için bir desen bulmamız gerekir.

`ab` okursak q_2'ye geliriz. q_2 kabul durumu değildir, ama boş geçişlerle q_0'a varırız ve q_0 kabul durumudur. Sonra yeni bir `ab` okuyabiliriz. Her `ab` serisinin sonuna, q_5'e gitmek için bir `aa` da eklenebilir. Bu yüzden `abaa` ve `ababaa` da dildedir.

Bu bilgiyi nasıl özetleriz? Önce döngüyü yazarız: $(ab)^*$. Ardından `aa` gelir. Hoca bunu küme gösterimiyle yazdı, çünkü iki kümeyi bitiştiriyoruz (concatenation):

$$(ab)^*\{aa\}$$

İlk küme `ab`, `abab`, `ababab` gibi dizgileri, ikinci küme yalnız `aa`'yı içerir. Elde ettiğimiz dizgiler `abaa`, `ababaa` gibidir. Hâlâ bir şey eksiktir: Sonuna `aa` gelmeyen dizgiler. Bunlar da küme olduğu için birleşimle (union) ekleriz. Dilin son hali şöyledir:

$$L(M) = (ab)^*\{aa\} \cup (ab)^*$$

Soru: Yalnız `aa`'nın olduğu durumu da birleşim olarak eklememiz gerekmiyor mu?

Cevap: Hayır, yalnız `aa` zaten kapsanmıştır. $(ab)^*$ yıldızlı olduğu için buraya boş dizgi gelebilir. Boş dizgi `aa` ile bitiştirilince sonuç yine `aa` olur.

Hocanın vurgusu: Bu kümeler biraz karmaşıklaşabilir. Bunları çalışmanız gerekir. Pratik yaptıkça görmek kolaylaşır.

Ders sonrası not: Hocanın ifadesi doğrudur. Bitiştirmenin birleşim üzerine dağılmasıyla aynı dil $(ab)^*(aa \cup \varepsilon)$ diye de yazılabilir. Python simülasyonu 14 sembole kadar bütün $\{a, b\}$ dizgilerinde makinenin dilinin bu ifadeyle aynı olduğunu gösterdi.

10. Makinelerin Denkliği

M_1 ve M_2 makinelerinin dilleri aynıysa bu makinelere denk (equivalent) denir:

$$M_1 \text{ ile } M_2 \text{ denktir} \iff L(M_1) = L(M_2)$$

Hoca örnek olarak dili $(10)^*$ olan makineyi M_1 aldı. Bu makineye denk birçok makine yazılabilir. Örneğin q_0 ve q_1 arasında yine `1` ve `0` geçişleri bulunsun, ama ayrıca `1` ile gidilen bir q_2 ölü durumu da olsun. q_2'ye gidilince bir şey olmaz. Bu geçerli bir makinedir ve dili de $(10)^*$'dır, yani M_1 ile M_2 denktir.

11. Teorem: NFA'lar Düzenli Dilleri Kabul Eder

Hoca bunu önemli bir teorem olarak tanıttı.

Teorem. NFA'ların kabul ettiği dillerin kümesi, düzenli dillerin (regular languages) kümesidir.

Düzenli dillerin DFA'ların kabul ettiği diller olduğunu hatırlayalım. Demek ki NFA'ların kabul ettiği diller, DFA'ların kabul ettiği dillerin aynısıdır. Bu yüzden DFA'lar ve NFA'lar aynı hesaplama gücüne (computation power) sahiptir. Bir makine daha karmaşık dilleri kabul edebiliyorsa hesaplama gücü daha yüksektir. Hoca bunun örneklerini ileride çok göreceğimizi söyledi.

11.1 İspatın Planı

İki kümenin eşitliğini ispatlamak için iki şey göstermemiz gerekir: Birinci küme ikincinin alt kümesidir (subset), ikinci küme de birincinin alt kümesidir. $\mathcal{L}_{NFA}$ NFA'ların kabul ettiği dillerin, $\mathcal{L}_{DFA}$ DFA'ların kabul ettiği dillerin kümesi olsun. Göstereceklerimiz şunlardır:

1. $\mathcal{L}_{NFA} \supseteq \mathcal{L}_{DFA}$
2. $\mathcal{L}_{NFA} \subseteq \mathcal{L}_{DFA}$

11.2 Birinci Kısım: $\mathcal{L}_{DFA} \subseteq \mathcal{L}_{NFA}$

Bu kısım önemsizdir (trivial), çok düşünmemiz gerekmez. Çünkü her DFA aynı zamanda bir NFA'dır.

Hoca a ve b geçişleri olan basit bir makine çizdi. Bu makine bir DFA'dır: ε geçişi yoktur, her durum ve her sembol için ne olacağı yazılmıştır ve her biri için tek geçiş vardır. Ama aynı zamanda bir NFA'dır. NFA'larda ε geçişi olabilir, ama olmak zorunda değildir. Her DFA aynı zamanda NFA olduğundan DFA'ların kabul ettiği dillerin kümesi, NFA'ların kabul ettiği dillerin alt kümesidir.

> **Ek bilgi (derste anlatılmadı):** Tanımlar tam olarak karşılaştırılırsa DFA'nın δ'sı bir durum, NFA'nınki bir küme verir. Bu yüzden DFA, NFA biçimine geçerken her $\delta(q, a) = p$ değeri $\{p\}$ olarak, her $\delta(q, \varepsilon)$ değeri $\emptyset$ olarak yazılır. Dil bu yazımla değişmez.

11.3 İkinci Kısım: $\mathcal{L}_{NFA} \subseteq \mathcal{L}_{DFA}$

Burada her NFA'nın denk bir DFA'sı olduğunu söylememiz gerekir. Elimizde ne kadar karmaşık, ε geçişli ve aynı sembolle birden çok duruma giden bir M_1 NFA'sı olursa olsun, $L(M_1) = L(M_2)$ olan bir M_2 DFA'sı bulunmalıdır. Önce her NFA'nın bir DFA'ya dönüştürülebileceğini göstermemiz gerekir. Bu gösterilirse, bir NFA'nın kabul ettiği her L dilini bir DFA da kabul eder.

> **Ders sonrası not:** Hoca burada önce "her DFA denk bir DFA'ya dönüştürülebilir" dedi ve hemen düzeltti. Doğrusu: Her NFA'nın denk bir DFA'sı vardır.

Hoca bu dönüşümün mümkün olduğunun ispatını derste atladı ve kalanının sonra okunabileceğini söyledi. Onun yerine dönüşümün nasıl yapılacağını örnekle göstermeye başladı.

12. NFA'dan DFA'ya Dönüşüm

Örnek üzerinden anlamak daha kolay olacak. M adlı NFA'nın üç durumu vardır:

$$q_0 \xrightarrow{a} q_1, \quad q_1 \xrightarrow{a} q_1, \quad q_1 \xrightarrow{\varepsilon} q_2, \quad q_2 \xrightarrow{b} q_0$$

Bu makineye karşılık gelen M' DFA'sını kuracağız. Başlangıç durumuyla başlarız, ama yalnız q_0 yazmak yerine içinde q_0 olan bir küme yazarız: $\{q_0\}$. Başlangıç durumumuz budur.

Neden küme? Çünkü NFA'da bir sembolle birden çok duruma gidilebilir. Bu çoklu duruma bir ad vermemiz gerekir. DFA'da her sembolde ne olacağını bilmek zorundayız, o halde bakalım.

$\{q_0\}$ **durumunda `a` gelirse:** q_1'e gideriz. Ama bir boş geçiş vardır, dolayısıyla doğrudan q_2'ye de gideriz ve makine kendini çoğaltır. NFA artık hem q_1 hem q_2 durumundadır. Bu iki durum bizim DFA'mızda tek bir durumla temsil edilmelidir. O durum ikisini temsil ettiği için adını $\{q_1, q_2\}$ koyarız. Bu küme, DFA'daki tek bir durumun adıdır. Adının içinde iki durum adı geçer.

$\{q_0\}$ **durumunda `b` gelirse:** q_0'da `b` geçişi yoktur. Hiçbir şey görmediğimiz için bir tuzak (trap) durumu oluşturmamız gerekir. `b` gelirse ölürüz. Burada ölü kalmak istediğimiz için bu duruma `a` ve `b` için kendine dönen bir döngü de ekleriz. Bu duruma istediğiniz adı verebilirsiniz. Hoca, böyle bir geçişin olmadığını anlatsın diye $\emptyset$ adını verdi.

Derste yapılan kısım şöyledir:

DFA durumu	a	b
$\{q_0\}$	$\{q_1, q_2\}$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$

Hoca, böyle düşünerek devam etmek gerektiğini söyleyip örneği burada bıraktı ve gelecek hafta sürdüreceğini belirtti.

> **Ek bilgi (derste anlatılmadı):** Örneğin kalan satırı aynı yöntemle bulunur. $\{q_1, q_2\}$'de `a` ile yalnız q_1'e gidilir ve boş geçişle sonuç yine $\{q_1, q_2\}$ olur. `b` ile yalnız q_2'den q_0'a gidilir ve sonuç $\{q_0\}$ olur. DFA'nın kabul durumları, NFA'nın kabul durumlarından en az birini içeren kümelerdir. Bu makinenin kabul durumu altyazıda söylenmediği için bütün boş olmayan F seçenekleri simülasyonla 11 sembole kadar denendi ve NFA ile DFA hep aynı sonucu verdi.
>
> **Ek bilgi (derste anlatılmadı):** DFA'nın her durumu Q'nun bir alt kümesi olduğu için durumlar, Bölüm 2.2'deki kuvvet kümesinden seçilir. Bu yüzden DFA'nın en fazla $2^{|Q|}$ durumu olur, yani durum sayısı her zaman sonludur. Bu örnekte $2^3 = 8$ sınırına karşılık yalnız 3 durum gerekti: $\{q_0\}$, $\{q_1, q_2\}$ ve $\emptyset$.

Dersin Toparlaması

Bu derste DFA'nın kuralları üç noktada gevşetildi: Aynı sembolle birden çok duruma gidilebilir, boş geçişler olabilir, bazı geçişler yazılmayabilir. NFA'nın geçiş fonksiyonu bu yüzden tek bir durum yerine kuvvet kümesinden bir küme verir. Bir NFA, en az bir hesaplaması bütün girişi okuyup kabul durumunda biterse dizgiyi kabul eder. Hesaplamaları görmek için hesaplama ağacı çizilir. Örneklerde $\{aa\}$, $(ab)^+$, $(10)^*$ ve $(ab)^*\{aa\} \cup (ab)^*$ dilleri bulundu.

NFA'lar dili ifade etmek için daha kolaydır, ama DFA'lardan daha güçlü değildir. İkisi de düzenli dilleri kabul eder. Teoremin $\mathcal{L}_{DFA} \subseteq \mathcal{L}_{NFA}$ kısmı her DFA'nın aynı zamanda NFA olmasından çıkar. Öbür kısım için hoca ispatı atlayıp dönüşüm yöntemini göstermeye başladı: DFA'nın durumlarına NFA durum kümelerinin adları verilir. Yarım kalan örnek gelecek hafta sürdürülecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.